```
from huggingface_hub import notebook_login

notebook_login()
```

```
!pip install -q -U transformers peft accelerate bitsandbytes pandas
!pip install -q pandas==2.2.2
```

```
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is
google-colab 1.0.0 requires pandas==2.2.2, but you have pandas 3.0.5 which is incompatible.
cudf-cu12 26.2.1 requires pandas<2.4.0,>=2.0, but you have pandas 3.0.5 which is incompatible.
dask-cudf-cu12 26.2.1 requires pandas<2.4.0,>=2.0, but you have pandas 3.0.5 which is incompatible.
```

```
from huggingface_hub import whoami

print(whoami())
```

```
{'type': 'user', 'id': '69f55c3555d9ad155ad24df4', 'name': 'ghargepratik', 'fullname': 'Pratik Bhanudas Gharge', 'canPay': F
```

```
!pip uninstall -y bitsandbytes
!pip install -U bitsandbytes==0.46.1
```

```
Found existing installation: bitsandbytes 0.50.1
Uninstalling bitsandbytes-0.50.1:
  Successfully uninstalled bitsandbytes-0.50.1
Collecting bitsandbytes==0.46.1
  Using cached bitsandbytes-0.46.1-py3-none-manylinux_2_24_x86_64.whl.metadata (10 kB)
Requirement already satisfied: torch<3,>=2.2 in /usr/local/lib/python3.12/dist-packages (from bitsandbytes==0.46.1) (2.11.0+
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from bitsandbytes==0.46.1) (2.0.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.2->bitsandbytes==0.46.1
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.2->bit
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.2->bitsandbytes==0
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.2->bitsandbytes==0
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.2->bitsandbytes=
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.2->bitsandbytes==0.46.1)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.2->bitsandbytes==0
Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cuc
Requirement already satisfied: cuda-bindings<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.2->bit
Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.2->
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.2-
Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.2->bits
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.2->bi
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.2->bitsandbytes==0
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toc
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolk
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolk
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolk
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolki
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolk
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cub
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings<13,>=12.9
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch<3,>=
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch<3,>=2.2->bitsa
Using cached bitsandbytes-0.46.1-py3-none-manylinux_2_24_x86_64.whl (72.9 MB)
Installing collected packages: bitsandbytes
Successfully installed bitsandbytes-0.46.1
```

```
BASE_MODEL = "meta-llama/Llama-2-7b-chat-hf"
```

```
import torch

from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    BitsAndBytesConfig
)

from peft import PeftModel


BASE_MODEL = "meta-llama/Llama-2-7b-chat-hf"
FINGPT_MODEL = "FinGPT/fingpt-forecaster_dow30_llama2-7b_lora"


# 4-bit configuration
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True
```

```
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True
)


print("Loading tokenizer...")

tokenizer = AutoTokenizer.from_pretrained(
    BASE_MODEL
)

if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token


# Install bitsandbytes if not available or outdated
try:
    import bitsandbytes
except ImportError:
    print("bitsandbytes not found or outdated, installing...")
    !pip install -U bitsandbytes>=0.46.1
    import bitsandbytes


print("Loading Llama 2...")

base_model = AutoModelForCausalLM.from_pretrained(
    BASE_MODEL,
    quantization_config=bnb_config,
    device_map="auto",
    torch_dtype=torch.float16
)


print("Loading FinGPT LoRA...")

model = PeftModel.from_pretrained(
    base_model,
    FINGPT_MODEL
)

model.eval()

print("FinGPT loaded successfully!")
```

```
Loading tokenizer...
Loading Llama 2...
[transformers] `torch_dtype` is deprecated! Use `dtype` instead!
Loading weights: 100%                                                 291/291 [00:55<00:00,  6.54it/s]
/usr/local/lib/python3.12/dist-packages/bitsandbytes/backends/cuda/ops.py:212: FutureWarning: _check_is_size will be removed
  torch._check_is_size(blocksize)
Loading FinGPT LoRA...
FinGPT loaded successfully!
```

```
!pip install -q flask flask-cors pyngrok
```

```
!pip install -q pyngrok
```

```
from pyngrok import ngrok

ngrok.set_auth_token("3I03QfMnKpgGkWYLJkTiBNiPXtU_5SLrRUbAUfrGMCmdPHNqS")
```

```
public_url = ngrok.connect(5000)

print(public_url)
```

```
NgrokTunnel: "https://unfunded-proven-caretaker.ngrok-free.dev" -> "http://localhost:5000"
```

```
!pip install -q feedparser flask flask-cors pyngrok
```

```
import feedparser
from urllib.parse import quote
from datetime import datetime, timedelta
from flask import Flask, request, jsonify
from flask_cors import CORS
import torch
```

```
# ============================================================
# NEWS FETCHING
# ============================================================

def get_india_news_by_topic(topic, days=7):

    cutoff = datetime.now() - timedelta(days=days)

    all_news = []

    # Google News is the most useful source for our demo.
    # The other RSS URLs may not work for every topic.
    sources = [
        (
            f"https://news.google.com/rss/search?"
            f"q={quote(topic)}+India"
            f"&hl=en-IN&gl=IN&ceid=IN:en",
            "Google News India"
        ),

        (
            f"https://economictimes.indiatimes.com/"
            f"rssfeeds/{quote(topic)}.cms",
            "Economic Times"
        ),

        (
            f"https://www.moneycontrol.com/rss/"
            f"{quote(topic)}.xml",
            "Moneycontrol"
        )
    ]

    for url, source_name in sources:

        try:

            feed = feedparser.parse(url)

            for entry in feed.entries:

                try:

                    if not hasattr(entry, "published_parsed"):
                        continue

                    published = datetime(
                        *entry.published_parsed[:6]
                    )

                    if published < cutoff:
                        continue

                    title = entry.get(
                        "title",
                        ""
                    ).strip()

                    link = entry.get(
                        "link",
                        ""
                    )

                    if not title:
                        continue

                    all_news.append({
                        "title": title,
                        "link": link,
                        "source": source_name,
                        "posted": published.strftime(
                            "%d %b %Y %I:%M %p"
                        ),
                        "published_datetime": published
                    })

                except Exception:
                    continue

        except Exception as e:
```

```
            print(
                f"News source failed: {source_name}",
                e
            )

            continue


    # Remove duplicate titles

    seen = set()

    unique = []

    for article in all_news:

        key = article["title"].lower().strip()

        if key not in seen:

            seen.add(key)

            unique.append(article)


    # Newest first

    unique.sort(
        key=lambda x: x["published_datetime"],
        reverse=True
    )


    # Remove internal datetime before returning

    for article in unique:

        article.pop(
            "published_datetime",
            None
        )


    return unique


# ============================================================
# FINGPT
# ============================================================

def run_fingpt(company, news_articles):

    # Take maximum 5 articles
    news_articles = news_articles[:5]

    news_text = ""

    for i, article in enumerate(news_articles):

        news_text += f"""
Article {i + 1}

Title:
{article["title"]}

Source:
{article["source"]}

Date:
{article["posted"]}

"""

    prompt = f"""
[INST]
<<SYS>>
You are a financial analyst.

Analyze the following recent news about the company.

Identify:
```

```
[Positive Developments]:
1.
2.

[Potential Concerns]:
1.
2.

[Analysis]:
Explain how the recent news may affect the company.

[Overall Impact]:
Positive / Negative / Neutral

Important:
Do not invent facts.
Base the analysis only on the provided news.
[/SYS>>

Company:
{company}

Recent News:
{news_text}

[/INST]
"""

    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=2048
    )

    inputs = {
        key: value.to(model.device)
        for key, value in inputs.items()
    }

    with torch.no_grad():

        output = model.generate(
            **inputs,
            max_new_tokens=350,
            do_sample=False,
            pad_token_id=tokenizer.eos_token_id
        )

    result = tokenizer.decode(
        output[0],
        skip_special_tokens=True
    )

    return result


# ============================================================
# FLASK API
# ============================================================

app = Flask(__name__)
CORS(app)


@app.route("/analyze-portfolio", methods=["POST"])
def analyze_portfolio():

    try:

        data = request.get_json()

        portfolio = data.get(
            "portfolio",
            []
        )

        if not portfolio:

            return jsonify({
                "error": "Portfolio is empty."
```

```
        }), 400


    results = []


    # ----------------------------------------
    # Process every stock in portfolio
    # ----------------------------------------

    for stock in portfolio:

        symbol = stock.get(
            "symbol",
            ""
        )

        company = stock.get(
            "name",
            symbol
        )

        quantity = stock.get(
            "quantity",
            0
        )

        print()
        print("=" * 60)
        print("Processing:", symbol)
        print("Company:", company)
        print("=" * 60)


        # Search using company name + symbol
        search_query = f"{company} {symbol}"


        # Fetch news
        news = get_india_news_by_topic(
            search_query,
            days=7
        )


        print(
            "News found:",
            len(news)
        )


        # FinGPT
        if news:

            analysis = run_fingpt(
                company,
                news
            )

        else:

            analysis = (
                "No relevant recent news "
                "was found."
            )


        results.append({

            "symbol": symbol,

            "company": company,

            "quantity": quantity,

            "news": news[:5],

            "analysis": analysis

        })
```

```
            return jsonify({

                "success": True,

                "results": results

            })


        except Exception as e:

            print("API ERROR:", e)

            return jsonify({

                "success": False,

                "error": str(e)

            }), 500


    print("Portfolio API ready.")
```

```
Portfolio API ready.
```

```
import threading

def run_server():

    app.run(
        host="0.0.0.0",
        port=5000,
        debug=False,
        use_reloader=False
    )


server_thread = threading.Thread(
    target=run_server,
    daemon=True
)

server_thread.start()

print("Flask server started on port 5000")
```

```
Flask server started on port 5000
 * Serving Flask app '__main__'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
```

```
import requests

response = requests.post(
    "http://127.0.0.1:5000/analyze-portfolio",
    json={
        "portfolio": [
            {
                "symbol": "TCS",
                "name": "Tata Consultancy Services",
                "quantity": 10
            }
        ]
    }
)

print("Status:", response.status_code)
print(response.text[:5000])
```

```
============================================================
Processing: TCS
Company: Tata Consultancy Services
============================================================
News found: 11
[transformers] Both `max_new_tokens` (=350) and `max_length`(=4096) seem to have been set. `max_new_tokens` will take precec
/usr/local/lib/python3.12/dist-packages/bitsandbytes/backends/cuda/ops.py:447: FutureWarning: _check_is_size will be removec
```

```
  torch._check_is_size(blocksize)
INFO:werkzeug:127.0.0.1 - - [17/Aug/2026 06:14:04] "POST /analyze-portfolio HTTP/1.1" 200 -
Status: 200
{"results":[{"analysis":"\n[INST]\n<<SYS>>\nYou are a financial analyst.\n\nAnalyze the following recent news about the comp
```